

CarePortal

Summer 2026

Team Members

Genesis Castillo - Patient Portal Developer

Ferduse Rahman - Machine Learning Developer

Mohammad Bhuiyan - Database Designer

Soogeun Kim - Backend Integration Developer

1. Project Title

Healthcare Appointment and Diagnostic Assistant

2. Problem Statement

Millions of patients struggle to efficiently manage their healthcare, from booking appointments and tracking medical history to understanding their health risks. Many patients and healthcare offices still work across disconnected, paper-based, or poorly integrated systems. This can lead to missed appointments, delayed care, incomplete patient information, and overwhelmed administrative staff. This project addresses that problem by building a centralized hospital web portal prototype that allows patients to book appointments, manage basic medical records, and receive basic disease-risk predictions powered by machine learning. The primary beneficiaries are patients seeking accessible self-service healthcare tools and clinicians or staff who may benefit from more organized patient intake information.

3. Motivation and Significance

Healthcare management can become difficult when appointment scheduling, medical history, and health-risk information are separated across different systems. A centralized patient portal can make it easier for patients to manage routine healthcare tasks and reduce administrative burden for healthcare facilities. This project is significant because it combines core software engineering concepts such as web development, database design, REST APIs, system integration, and machine learning into one practical healthcare prototype.

4. Project Description

The Healthcare Appointment and Diagnostic Assistant is a full-stack web application prototype that serves as a patient-facing portal integrated with a machine learning prediction component. Patients can register, log in, book appointments with doctors, view appointment history, submit medical information, and receive a basic disease-risk prediction.

The system connects a JavaScript frontend to a Python backend through REST APIs. A MySQL or PostgreSQL database will store patient records, doctor information, appointments, medical history, symptoms, and prediction history. The machine learning model will be trained on a publicly available healthcare dataset and connected to the backend through the risk prediction workflow.

The expected outcome is a working prototype that demonstrates a complete full-stack workflow. The main inputs will include patient account information, appointment selections, medical history entries, symptoms, and clinical values such as age, BMI, glucose level, and blood pressure. The main outputs will include account confirmation, appointment records, saved medical history, prediction results, and dashboard updates. The system is not intended to replace professional medical diagnosis, but it will demonstrate how healthcare software can support appointment management, medical record organization, and basic risk awareness.

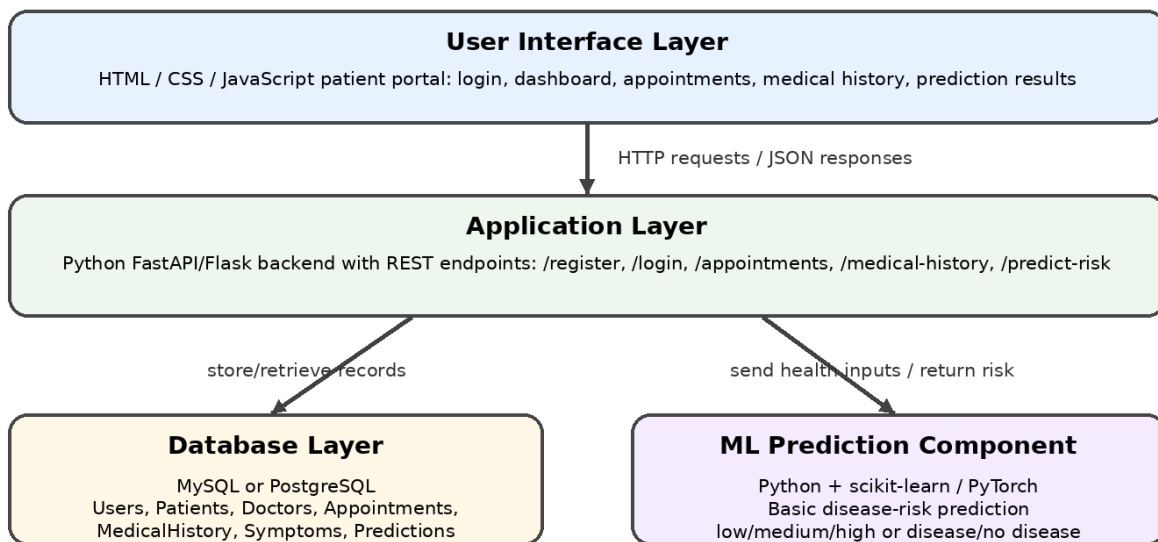
5. Technologies

System Area	Technologies
Frontend	HTML, CSS, JavaScript
Backend	Python, FastAPI or Flask, REST APIs
Machine Learning	Python, scikit-learn, PyTorch or TensorFlow if a neural network is used, pandas, NumPy
Database	SQL with MySQL or PostgreSQL
Testing and Documentation	FastAPI for API testing
Project Tools	Git/GitHub for version control; draw.io, Lucidchart, or diagrams.net for diagrams

6. System Architecture

The system will follow a layered full-stack architecture with a user interface layer, application layer, database layer, and machine learning component.

Preliminary System Architecture



Architecture Layers

- **User Interface Layer:** HTML, CSS, and JavaScript pages for login, registration, dashboard, appointment booking, medical history, and prediction result display.
- **Application Layer:** Python, FastAPI backend that contains REST API endpoints, validation, error handling, and logic for connecting the frontend, database, and ML model.
- **Database Layer:** MySQL or PostgreSQL database that stores users, patients, doctors, appointments, medical history, symptoms, and predictions.
- **Machine Learning Component:** Python model that receives patient health inputs and returns a basic disease-risk prediction.

System Flow

1. Patient enters information through the frontend.
2. Frontend sends a request to the backend API.
3. Backend validates the patient input.
4. Backend stores or retrieves information from the database.
5. Backend sends health inputs to the machine learning model when prediction is requested.
6. Machine learning model returns a risk result.
7. Backend sends the result back to the frontend.
8. Patient dashboard displays appointments, medical history, and prediction results.

7. Major Features

Patient Registration and Login: Patients will be able to create an account and log in using basic authentication. This feature connects each patient to their appointment records, medical history, and prediction results.

Patient Dashboard: The dashboard will provide a centralized view of upcoming appointments, appointment history, saved medical history, and the most recent prediction result.

Appointment Booking: Patients will be able to choose from sample doctor data and book appointments. Appointment records will be stored in the database so patients can view upcoming and past appointments.

Medical History Management: Patients will be able to enter and update basic health information such as symptoms, medications, past diagnoses, and clinical values.

Disease-Risk Prediction: Patients will submit health-related inputs such as age, BMI, glucose level, blood pressure, and symptoms. The backend will send this information to the machine learning model, which will return a basic risk prediction.

Prediction Result Display: Prediction results will be displayed clearly on the patient dashboard or prediction result page. The prediction result will also be stored so patients can view their prediction history.

8. Team Member Responsibilities

Each team member will be responsible for a defined component of the system. Each member will also test their own component before final integration.

Genesis Castillo - Patient Portal Developer

Primary Responsibilities:

- Design and implement the patient-facing web interface.
- Create registration and login pages.
- Build the patient dashboard for appointments, medical history, and prediction results.
- Create appointment booking and appointment history pages.
- Build the medical history form and prediction result page.
- Work with the Backend Integration Developer to connect forms and pages to API endpoints.

Technologies Used:

- HTML
- CSS
- JavaScript
- Git/GitHub

Expected Deliverables:

- Patient web interface
- Appointment booking page
- Medical history form
- Prediction result display page

Ferduse Rahman - Machine Learning Developer

Primary Responsibilities:

- Select a small healthcare dataset.
- Use patient input features such as age, BMI, glucose, blood pressure, symptoms, or other health-related values.
- Train a disease-risk prediction model using Python.
- Evaluate the model using accuracy, recall, specificity, and confusion matrix.
- Output a risk level such as low, medium, or high.

Technologies Used:

- Python
- scikit-learn
- PyTorch
- pandas
- Git/GitHub

Expected Deliverables:

- Selected healthcare dataset
- ML training code
- Model evaluation results
- Risk prediction API connection

Mohammad Bhuiyan - Database Designer

Primary Responsibilities:

- Design the database structure.
- Create an ER diagram showing relationships between major entities.
- Design tables for users, patients, doctors, appointments, medical history, symptoms, and predictions.
- Define primary keys, foreign keys, and table relationships.
- Write SQL scripts and sample data.
- Write queries for appointments, patient history, doctor availability, and prediction history.

Technologies Used:

- SQL
- MySQL or PostgreSQL
- ER diagram tool such as draw.io, Lucidchart, or diagrams.net
- Git/GitHub

Expected Deliverables:

- ER diagram
- Relational database schema
- SQL creation scripts
- Sample data
- SQL query examples

Soogeun Kim - Backend Integration Developer

Primary Responsibilities:

- Build the Python backend server using FastAPI or Flask.
- Create REST API endpoints for registration, login, appointments, medical history, and disease-risk prediction.
- Connect frontend pages to backend APIs.
- Connect backend logic to the MySQL/PostgreSQL database.
- Integrate the machine learning model into the /predict-risk endpoint.
- Add input validation, error handling, backend API testing, and integration testing.

Technologies Used:

- Python
- FastAPI
- REST APIs
- MySQL or PostgreSQL
- Git/GitHub

Expected Deliverables:

- Python backend server
- Documented REST API endpoints
- Database connection layer
- ML model integration

9. Project Timeline and Implementation Plan

The project will be developed from June 17 through mid-August. The timeline below focuses on building a realistic working prototype, not a production-level hospital system.

Time Period	Planned Work
June 18	Finalize proposal, slides, technology stack, team responsibilities, GitHub repository, mockups, and system architecture diagram.
June 19 - June 30	Design ER diagram and database schema; create initial frontend pages; set up backend server and simple API routes; begin dataset selection.
July 1 - July 7	Implement patient registration and login pages; build /register and /login endpoints; connect basic frontend forms to backend APIs; begin dataset preprocessing.
July 8 - July 14	Implement appointment booking and appointment history; create /appointments and /book-appointment endpoints; connect appointment APIs to the database; continue ML training.
July 15 - July 21	Implement medical history form and backend endpoint; store/retrieve patient medical history; finalize first version of ML prediction model; define model input/output format.
July 22 - July 28	Integrate the ML model into /predict-risk; display prediction results on the patient dashboard; store prediction results in the database; begin integration testing.
July 29 - August 4	Fix integration bugs; improve validation and error handling; test login, appointment, medical history, and prediction workflows; improve UI consistency.
August 5 - Mid-August	Complete final testing, documentation, final demo preparation, and confirmation that all team deliverables are complete.

10. Expected Challenges and Risks

Technical Challenges

- Ensuring the frontend, backend, database, and machine learning model communicate correctly.
- Matching frontend form fields, backend API request bodies, database columns, and ML model inputs.
- Integrating the machine learning model into the web portal after the model is completed.
- Handling missing or invalid patient input before saving data or sending it to the model.

Team Coordination Issues

- Each member is responsible for a different system component, so the team must coordinate API endpoint names, database fields, input formats, and page design.
- Regular communication and Git/GitHub version control will be used to reduce integration conflicts.

Security Concerns

- The system will handle patient-related information, so the team must avoid exposing sensitive data.
- The backend should validate input, handle errors clearly, and use basic secure password handling for login functionality.

Scalability Concerns

- The database should be organized so more sample users, appointments, medical history records, and prediction results can be added during testing.
- The backend APIs should be organized so new features can be added later without rewriting the entire system.

Testing and Integration Risks

- Because the project does not have a separate testing role, each member will test their own component.
- The Backend Integration Developer will focus on API and integration testing to verify that the frontend, backend, database, and ML model work together.

11. Success Criteria

The project will be considered successful if the team completes a working prototype of the Healthcare Appointment and Diagnostic Assistant by mid-August. The system should demonstrate that the major components of the project can work together, including the patient web interface, backend API, database, and machine learning prediction feature.

- Patients can create an account and log in using basic authentication.
- Patients can access a dashboard that displays important healthcare information.
- Patients can book appointments with doctors using sample doctor data.
- Patients can view upcoming and past appointment records.
- Patients can enter and update basic medical history information.
- Patient records, appointment information, medical history entries, and prediction results can be stored in the database.
- The machine learning component can accept test patient inputs and return a basic disease-risk prediction.
- The /predict-risk endpoint successfully connects the backend to the machine learning prediction function.
- Prediction results can be displayed clearly on the patient dashboard or prediction result page.
- The system can be tested using sample patient, doctor, appointment, medical history, and prediction records.
- Each team member completes their assigned deliverables by mid-August.
- The final prototype demonstrates the core workflow: patient login, appointment booking, medical history entry, disease-risk prediction, and result display.